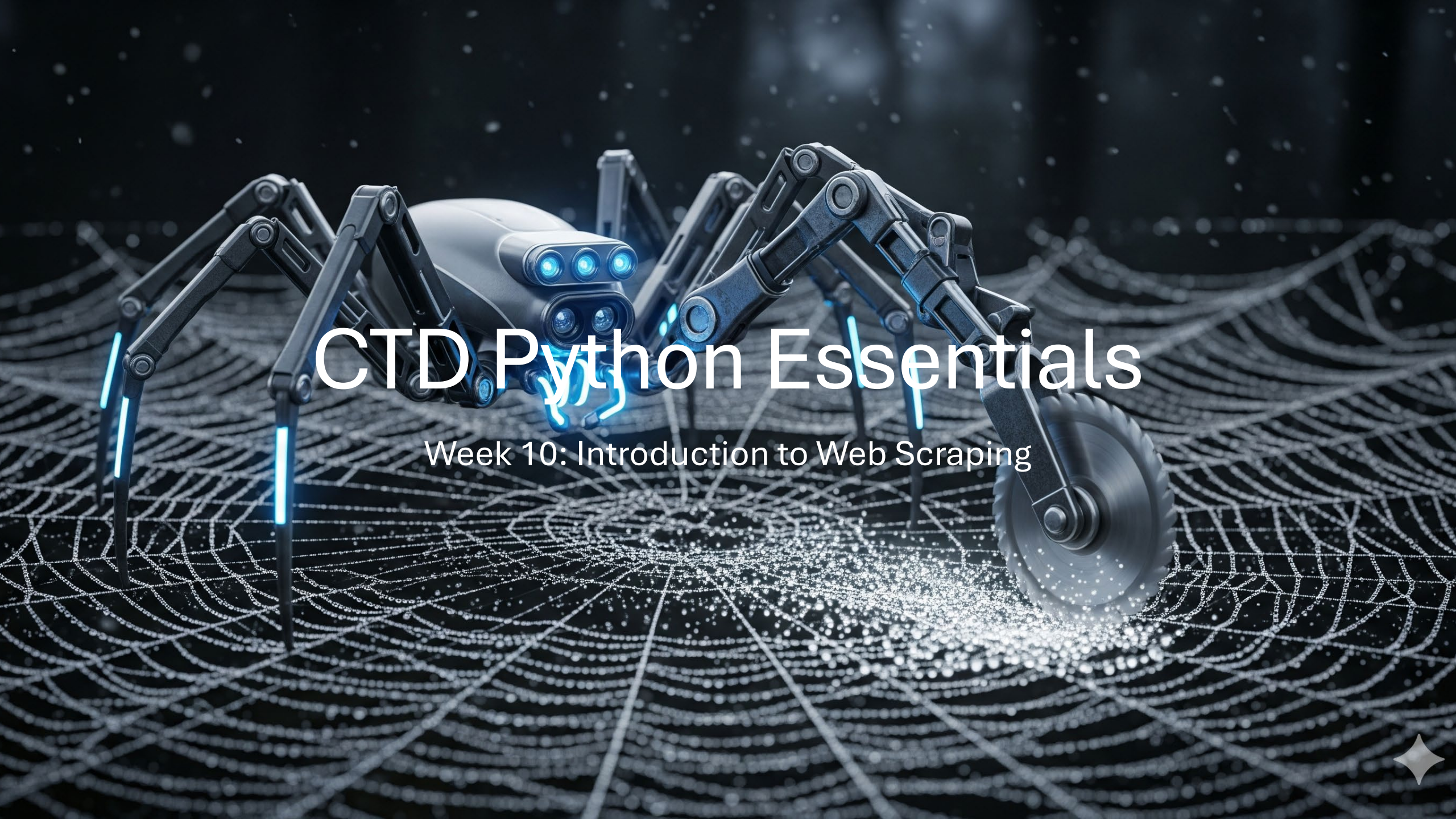


A futuristic robotic spider with a dark, metallic body and eight articulated legs. The spider has four glowing blue eyes and several blue light strips on its legs. One of its legs is replaced by a large, circular saw blade. It is standing on a complex, glowing web structure that resembles a network or data map. The background is dark with floating particles.

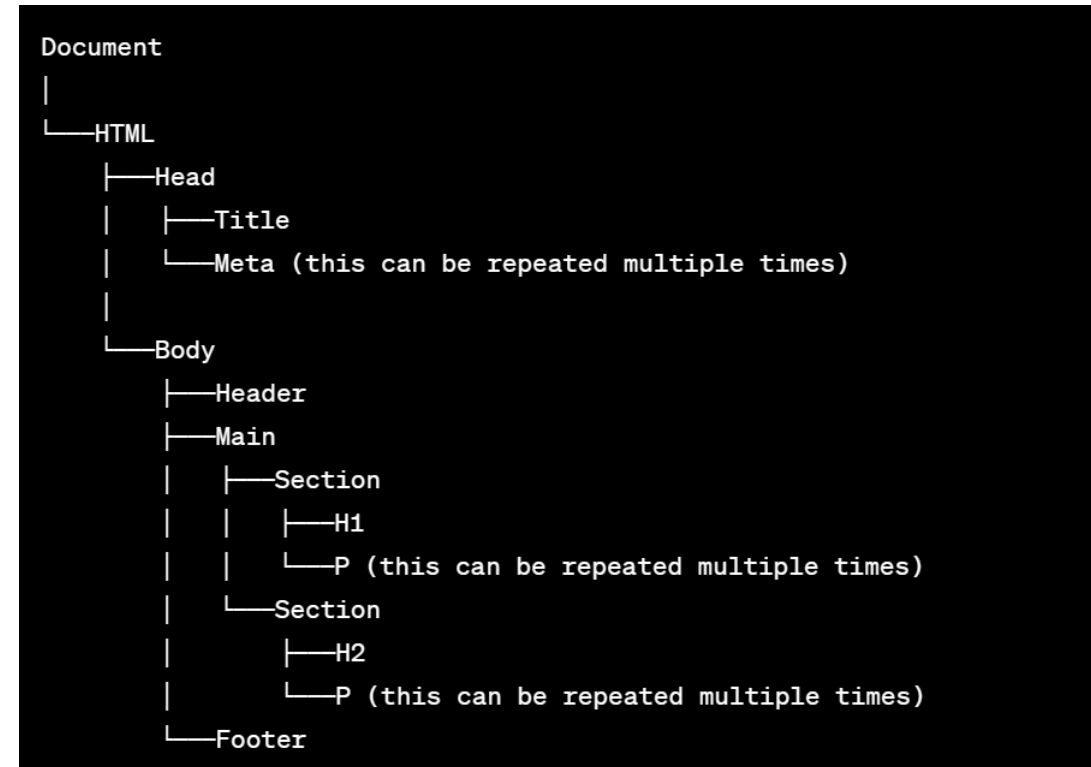
CTD Python Essentials

Week 10: Introduction to Web Scraping



HTML and DOM Review

- HTML is a semantic markup language
 - Tags and elements define semantics and are not displayed
 - Most elements have opening and closing tags
 - `<p>A really short paragraph.</p>`
 - Some are self-closing single tags
 - ``, `<br>`, `<meta>`
 - Attributes
 - Add directives and information to tags
 - `<a href="link-path">`
 - `<div id="unique-identifier">`
 - `<p class="treat-everything-in-this-class-the-same">`
- An HTML document is stored in a tree structure known as the Document Object Model (DOM)



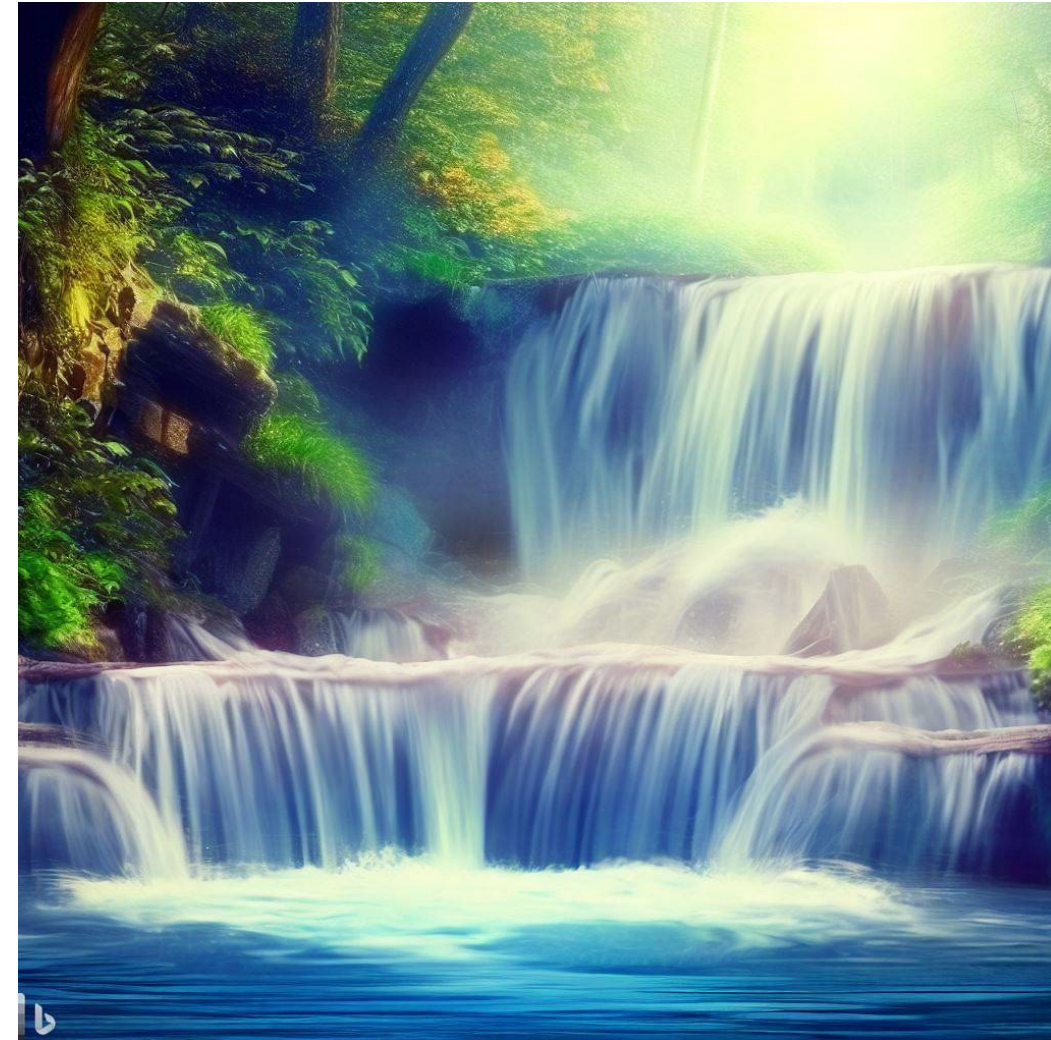
Selenium and Webdriver

- [Selenium](#) is a powerful framework for web scraping
 - Supports dynamic content
 - Almost all modern web pages have dynamic content generated by javascript frameworks
 - Usually use data from fetch or ajax calls
- A dynamic scraping framework needs to simulate a web browser
 - Selenium works in concert with [Webdriver](#)
- Static scraping frameworks won't work for dynamic pages
 - [beautifulsoup4 · PyPI](#) is an example of a static scraping framework

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.options import Options
from selenium.common.exceptions import TimeoutException, NoSuchElementException
import time
import json
import csv
```

CSS Selectors

- Type (p, h1,...)
- Class <p class=foo> => .foo { ... }
- Id <p id=special> => #special { ... }
 - Id's are unique to the entire document
- Types can be combined with class or id: p.foo
- Id takes precedence over class
- Descendant selectors
 - Elements that are descendants of another element
 - e.g. li a => any a element nested inside an li element
 - With an id: #unique a => any a element nested inside the id=unique element
 - Multiple levels: #nav ul li a
- Universal: * => everything
 - Also descendant: p *
- Child selector:
 - div > p => a paragraph which is a child of a div
- Attribute selectors:
 - Img[src="small.jpg"]



Using By Selectors in Selenium

- from `selenium.webdriver.common.by` import `By`
 - `driver.find_element(By.CSS_SELECTOR, any-css-selector)`
- Other `By` selectors
 - ID
 - CLASS_NAME
 - LINK_TEXT
 - PARTIAL_LINK_TEXT
 - NAME
 - TAG_NAME
 - XPATH

```
quotes = wait.until(EC.presence_of_all_elements_located((By.CLASS_NAME, "quote")))  
next_button = driver.find_element(By.PARTIAL_LINK_TEXT, "Next")  
submit_button = driver.find_element(By.CSS_SELECTOR, "input[type='submit']")  
password_field = driver.find_element(By.ID, "password")  
submit_button = driver.find_element(By.XPATH, "//div[@class='btn' and @type='submit']")
```

Xpath selectors

- Sometimes CSS selectors aren't sufficient
- Xpath selectors can help when, for example:
 - Elements don't have unique IDs or classes
 - Selection is based on visible text
 - Finding elements using relative selection
 - Handling dynamically generated content where other selectors break

Path Expressions

Expression	Description	Example
/	Absolute path from root	/html/body/div
//	Relative path (anywhere in document)	//div
.	Current node	./span
..	Parent node	../div
@	Attribute selector	@class

Node Selection

Expression	Description	Example
tagname	Select all elements with tag	//div
*	Select all elements	//*
@attribute	Select attribute	//@class
node()	Select all nodes	//node()
text()	Select text content	//div/text()

- `//div` - selects all div elements anywhere in document
- `/html/body/div` - absolute path from root
- `//div[@class='example']` - div with specific class
- `//input[@id='username']` - input with specific ID
- `//span[contains(text(), 'Click here')]` - element containing specific text

Managing Requests

- Timing, errors, and exceptions need to be handled during web scraping.
- It's important to wrap scraping requests with try blocks
 - Selenium provides some helpful exceptions
 - `NoSuchElementException`
 - `TimeoutException`
- Selenium's `WebDriverWait` provides convenient capabilities for managing loading time and events.
 - Waits only as long as necessary and immediately proceeds when a specific condition is met

```
# Wait up to 10 seconds for quotes to appear, proceed immediately when found
wait = WebDriverWait(driver, 10)
quotes = wait.until(EC.presence_of_all_elements_located((By.CLASS_NAME, "quote"))))

# Wait for next button to be clickable before clicking
next_button = wait.until(EC.element_to_be_clickable((By.PARTIAL_LINK_TEXT, "Next"))))
next_button.click()
```

Web Scraping Ethics

- The [robots.txt](#) file is a standard part of a web site which is used to specify permissions for automated processes such as web scraping.
- It can always be accessed at '/robots.txt' from the top level of the web site.
- It specifies portions of the site which don't allow automated access (Disallow)
- Specific User-agents can be allowed or excluded
- Allow everything:

```
User-agent: *  
Disallow:
```
- It's also important to avoid code which might overload a site
 - For example, repeated loading of the same content
- Detailed [guide to robots.txt](#)

```
User-agent: Googlebot  
Disallow: /
```

Scraping Robustness

- Web scraping is inherently unstable and prone to high maintenance
- Web site owners don't generally retain the same DOM structure across revisions
- It may be possible to make code somewhat independent of structure
 - Avoid depending on hierarchy details
- In general APIs are a superior way to obtain data
 - They imply that the information is being shared explicitly



Q&A and Demo

Gist with demo code:

[CTD Python 100 Lesson 10 Code Examples](#)

Scraping from: [Quotes to Scrape](#)

For more practice, check out: [Books to Scrape](#)

